BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 03

> **Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

> Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.

4. Add three new keys to the dictionary to pass the global state to the agent:
   `'grid_size': (self.width, self.height)`
   `'walls': list(self.walls)`
   `'all_food': list(self.food_positions)`

Step 1.2: Implementing BFS, DFS, and UCS

> You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

> The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

State Space: The abstract graph of all valid configurations (states) and allowable transitions defined by the environment. In our grid game, the state space is finite, consisting of the unique grid coordinates (x, y) bounded by the grid dimensions.
Search Tree: An explicit tree generated dynamically during the search process starting from the initial state as the root. Nodes in the search tree represent paths through the state space. A single state in the state space can appear as multiple distinct nodes across different branches of the search tree. Unlike the state space, a search tree can grow infinitely large if loops exist.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

Problem Solved: The reached (or visited) set converts a Tree Search into a Graph Search. It prevents duplicate state expansion by recording every state that has already been explored or enqueued.

Effect on DFS without reached: Without a reached set, DFS will get trapped in infinite loops Because DFS always expands the deepest node first, it will follow an infinite branch forever, resulting in an infinite loop or stack overflow without ever finding the food pellet.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS Optimality: BFS expands nodes level-by-level in order of increasing depth (shallowest nodes first). In a grid where each step (Up, Down, Left, Right) has a uniform step cost of 1, path depth equals total path cost . Therefore, the first time BFS pops the goal state, it is guaranteed to have found the path with the minimum number of steps.

DFS Suboptimality: DFS follows a single branch down to its deepest point before backtracking. It picks a direction (e.g., favoring Right or Down) and follows it as far as possible until it hits a wall or boundary, often driving the agent far away from a nearby goal and resulting in long, erratic, serpentine paths.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS has a **high memory requirement** because it stores all frontier nodes at the current depth in the queue. Its space complexity is **O(b^d)**, which is exponential. In a massive **1000 × 1000 grid**, the number of stored nodes can grow very quickly, causing high RAM usage and possibly an **Out-of-Memory (OOM)** crash before finding the food.

In contrast, DFS has a **much lower memory requirement** because it mainly stores the current path from the starting node to the deepest node, along with some unexplored nodes. Its space complexity is **O(bm)** or **O(m)**, which grows linearly with the maximum search depth. Therefore, DFS is much less likely to run out of memory than BFS on a very large grid.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

FACULTY OF COMPUTING